

CSE 654 / 484 - Fall 2024

Homework 1

Sümeyye Zeynep Gürbüz

200104004039

1. Introduction

N-gram language models are statistical language models that use sequences of "n" items (grams) from a given dataset to predict the next item in a sequence based on the preceding items. These models are particularly useful in natural language processing tasks, such as speech recognition and text generation, as they allow for the estimation of probabilities for sequences of words or characters.

In this homework, I developed two distinct statistical language models for the Turkish language: a syllable-based model and a character-based model. The syllable-based model focuses on predicting sequences based on syllable units, while the character-based model operates on individual character units. The primary objectives of this homework include calculating N-gram probabilities, evaluating perplexity, and generating random sentences using these models.

Through the analysis of generated sentences, we aim to explore the quality and fluency of sentences based on the chosen N-gram level (1-Gram, 2-Gram, and 3-Gram) and the underlying model type (syllable-based vs. character-based).

2. Design and Implementation

Code Structure Overview:

The code is structured to perform the following key tasks:

- **Data Loading and Preprocessing:** Load the dataset, extract relevant content, and preprocess the text to separate syllables and characters.
- **N-gram Creation:** Generate frequency tables for 1-grams, 2-grams, and 3-grams.
- **Good-Turing Smoothing:** Apply smoothing to the N-gram counts to adjust the probabilities and handle zero-frequency problems.

- **Perplexity Calculation:** Evaluate the performance of the language models based on perplexity metrics.
- **Random Sentence Generation:** Generate random sentences from the trained models using the N-grams.

Detailed Function Descriptions:

1. `load_data(file_path)`

- Input: The path to the text file to be loaded.
- Output: The content of the file as a string.
- Functionality: Opens the specified file, reads its content, and returns it for further processing.

2. `extract_data(content)`

- Input: A string containing the content of the loaded data.
- Output: A string of extracted articles, each extracted from their id, url, title, body.
- Functionality: Uses regular expressions to extract articles from the content based on predefined tags.

3. `split(data)`

- Input: A string of articles.
- Output: Two lists: `train_data` and `test_data`.
- Functionality: Splits the 70% of dataset into training and testing sets.

4. `tokenize_sentences(data)`

- Input: A string representing a single document.
- Output: A list of tokenized sentences.
- Functionality: Utilizes the `TrnlpToken` class to segment the input text into sentences.

5. `tokenize_syllables(word)`

- Input: A string representing a single word.
- Output: A list of syllables derived from the word.
- Functionality: Uses the syllabification function from the `Trnlp` library to break words into syllables.

6. `clean(data)`

- Input: A string of text.

- Output: A cleaned string of text with URLs and escape characters removed.
- Functionality: Removes unwanted characters from the text using regular expressions.

7. preprocess(data)

- Input: A list of articles or a single string.
- Output: Two lists: one containing syllables and the other containing characters.
- Functionality: First cleans the input data. After that, tokenizes it into words, and extracts syllables for the syllable-based model; also normalizes and extracts characters for the character-based model.

8. create_ngram_tables(tokens)

- Input: A list of tokens (syllables or characters).
- Output: Three lists containing unigrams, bigrams, and trigrams.
- Functionality: Uses the ngrams function to generate the frequency tables of N-grams from the token list.

9. apply_good_turing_smoothing(frequency_counts)

- Input: A Counter object containing N-gram frequencies.
- Output: A dictionary of smoothed probabilities.
- Functionality: Implements Good-Turing smoothing to adjust N-gram probabilities and handle zero-frequency issues.

10. calculate_perplexity(test_data, ngram_probs, n)

- Input: The test data, N-gram probabilities, and the value of n (1, 2, or 3).
- Output: A calculated perplexity score.
- Functionality: Evaluates the perplexity of the model based on the logarithmic probability of the N-grams in the test data.

11. generate_random_sentences(ngram_model, n, sentence_length)

- Input: The N-gram model, the value of n, and the desired sentence length.
- Output: A randomly generated sentence as a string.
- Functionality: Generates a random sentence by selecting from the top 5 N-grams at each step, extending the sentence based on the selected N-grams.

Main Function for N-gram Language Model Implementation:

The main function executes the process of building both syllable-based and character-based N-gram language models, from loading the data to calculating perplexities and generating sample sentences. Each step is modularized for clarity and efficiency.

1. Data Loading

- **Function Call:** `load_data('wiki_00')`
 - Loads the content of the `wiki_00` file and reads it as a single string.

2. Data Extraction

- **Function Call:** `extract_data(content)`
 - Processes the loaded data to extract individual articles based on specified patterns, returning a list of parsed articles.

3. Data Splitting

- **Function Call:** `split(extracted_content)`
 - Splits the (70% of) extracted content into a training set and a testing set (95% training, 5% testing).

4. Data Preprocessing

- **Function Calls:** `preprocess(train_data)` and `preprocess(test_data)`
 - Separately processes both the training and testing data for syllable and character-based tokenization, using tokenization methods (`TrnlpToken` and `trnlp.helper.syllabification`) to yield syllable and character lists.
 - Returns syllable and character lists, which are required for N-gram generation.

5. N-gram Frequency Table Creation

- **Function Calls:** `create_ngram_tables(train_syllables)` and `create_ngram_tables(train_chars)`
 - Constructs frequency tables for syllables and characters, generating unigram, bigram, and trigram lists.

6. Counting N-grams

- **Code:** `Counter(syll_unigrams), Counter(syll_bigrams), Counter(syll_trigrams)` for syllables and `Counter(char_unigrams), Counter(char_bigrams), Counter(char_trigrams)` for characters.
 - Counts the occurrences of each N-gram using `Counter`, producing dictionaries that map N-grams to their frequencies.

7. Good-Turing Smoothing

- **Function Calls:** `apply_good_turing_smoothing(syll_uni_counts)`, `apply_good_turing_smoothing(syll_bi_counts)`, etc.
 - Applies Good-Turing smoothing to each N-gram frequency dictionary, adjusting counts for unseen or rare N-grams and outputting smoothed probability tables.

8. Perplexity Calculation

- **Function Calls:** `calculate_perplexity(test_syllables, syll_uni_gtp, 1)`, `calculate_perplexity(test_chars, char_uni_gtp, 1)`, etc.
 - Computes the perplexity of each smoothed N-gram model against the test set, providing a measure of how well the model predicts the test data.

9. Output of Model Perplexities

- **Code:** `print(...)`
 - Displays the calculated perplexities for syllable-based and character-based models in a structured format.

10. Sentence Generation

- **Function Calls:** `generate_random_sentences(syll_uni_counts, 1, 20)`, `generate_random_sentences(char_bi_counts, 2, 20)`, etc.
 - Uses the N-gram probability models to generate sentences of 20 tokens for each of the N-gram models (unigrams, bigrams, and trigrams).
 - Outputs generated sentences for both syllable-based and character-based models.

3. Results and Tables

Model Type	N-gram	Perplexity
Syllable-Based	1-gram	1.000002
Syllable-Based	2-gram	1.000003
Syllable-Based	3-gram	1.000003
Character-Based	1-gram	1.000000
Character-Based	2-gram	1.000000
Character-Based	3-gram	1.000001

Model Type	N-gram	Generated Sentence
Syllable-Based	1-gram	de
	2-gram	i le doğ muş bu ba ba ba ka tıl mak ko mis yo ğun da en cid di Ni
		o la nan Şük rü cü ol du ğu un gın la ra sı ra sı o tur se E
		le ri qu es tor Ta pes Top la rak Pa so nu ken di a ra sın ken Se
	3-gram	ner bah çe Sa ra coğ lu bir spor a ra yüz ler dir İlk de fa Wın dows 'ta
		o la rak Ga zi Mus ta fa Çak mak 'ın ne za re ti'n de ya rı bu hiz
Character-Based		ta ra fin Ta nı tıl mış ol duk la rıy mış Ör güt kıs mi tü re bi rey
	1-gram	i
		e
	2-gram	a r m a b a t i ' , 9 0 6 ' ' m a
		l a h i l ü . s i l ü b a h i y e y i
		a ' , 3) . k t e r n d y e n e n d e
	3-gram	a n n t ı ş b e d m) ' d ü n ü ' ,
		v e t u l ü p t ü n l e r i s k f y
		d a " p o z a d e h i r ' i s t a k v

4. Analysis and Conclusion

Performance Analysis Based on Perplexity Values

The perplexity values for the syllable-based and character-based models are as follows:

- **Syllable-Based Model:**
 - 1-gram: 1.000002
 - 2-gram: 1.000003
 - 3-gram: 1.000003
- **Character-Based Model:**
 - 1-gram: 1.0000004

- 2-gram: 1.0000005
- 3-gram: 1.0000005

In language modeling, lower perplexity typically implies better predictive performance. Both models show very low perplexity values, suggesting strong predictive accuracy. However, the character-based model has slightly lower perplexity values than the syllable-based model, which may indicate a marginally better performance in predicting sequences. This result aligns with the fact that character-based models can capture finer details, given the larger dataset they represent compared to syllables.

Trade-Offs Between Syllables and Characters for N-grams

- **Syllable-Based Model:**
 - Working at the syllable level aligns more closely with Turkish linguistic structure, as Turkish words are often constructed from clear syllabic units due to agglutination. This model can better capture morphemes and thus grammatical relationships, improving coherence at the word and sentence level.
 - However, because the syllable-based model represents fewer units than the character-based model, it may lack flexibility in capturing certain nuances in text structure, particularly in short or frequently used syllables.
- **Character-Based Model:**
 - Character-based modeling is highly granular, which enhances its ability to capture variations at the letter level, especially in languages with rich vowel harmony and consonant pairings, like Turkish.
 - While the character-based model shows lower perplexity, the generated sentences are less coherent than those from the syllable model due to the lack of semantic grouping. In other words, while it may predict individual letters well, it struggles with meaningful syllabic or word-level predictions.

In summary, while character-based models may yield better perplexity scores, syllable-based models might offer more coherent sentence generation.

Quality of Generated Sentences Across N-gram Levels and Models

- **1-gram Sentences:** Both models perform poorly in generating meaningful sentences at the 1-gram level, as individual syllables or characters alone lack

the context needed for coherent sentence formation. Most outputs are simply isolated syllables or characters.

- **2-gram Sentences:**

- The syllable-based model demonstrates improved structure, yielding phrases that appear as partial or fragmented sentences, though not fully coherent. This suggests that syllable-based 2-grams can capture some degree of word-level context.
- The character-based model generates sequences that are largely nonsensical or seem like random collections of letters and punctuation, indicating that it lacks the structure for useful sentence formation at this level.

- **3-gram Sentences:**

- In the syllable-based model, 3-grams produce outputs that are closer to meaningful phrases, with recognizable patterns that suggest plausible Turkish sentence structure, though they remain far from fluent. This implies that 3-gram syllable-based modeling can capture some morphemic or grammatical structures.
- The character-based 3-gram sentences remain random and lack coherence, highlighting the limitations of character-level 3-grams in modeling Turkish morphology and sentence structure.

Conclusion: Optimal Model and N-gram Size for Turkish

Based on the analysis, the **syllable-based model with 3-grams** appears to be the most suitable for modeling Turkish. Despite its slightly higher perplexity values compared to the character-based model, it generates more coherent and structurally appropriate sentences, which better reflects the morphological and grammatical characteristics of Turkish.

In essence, while the character-based model performs well in terms of perplexity, it lacks the capacity for coherent generation due to Turkish's syllable-centric structure. Therefore, the syllable-based 3-gram model strikes a good balance between predictive accuracy and linguistic coherence, making it a more suitable choice for tasks requiring the generation of meaningful Turkish text.

5. Table on LLM Usage

Section	Source of Assistance
Text Preprocessing	Got help for Python syntax from ChatGPT
Data Splitting	Got help for syntax from ChatGPT
N-gram Calculation	Used both ChatGPT and internet resources
Smoothing	Used both ChatGPT and internet resources
Perplexity Calculation	Used ChatGPT
Random Sentence Generation	Used ChatGPT
Report	Used ChatGPT for the main structure